

Foundations of Deep Learning

Lecture 10

REINFORCEMENT LEARNING

Aurelien Lucchi

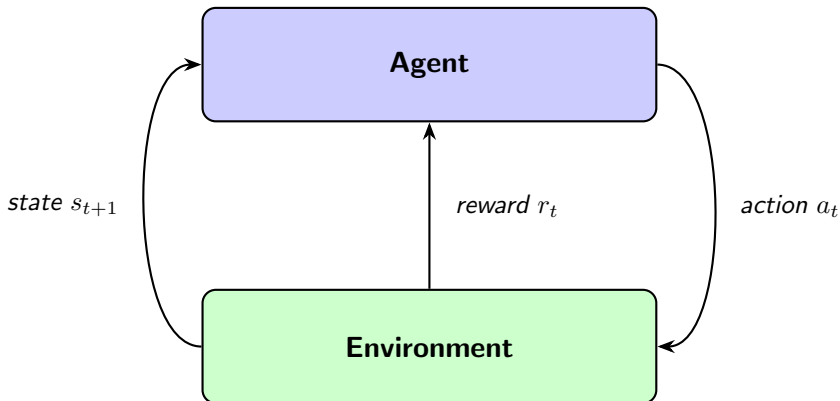
Fall 2025

What is Reinforcement Learning (RL)?

Definition RL is a paradigm in which an agent learns to make a **sequence of decisions** by interacting with an environment and **receiving feedback** in the form of rewards to maximize its long-term cumulative return.

Deep RL = RL + Deep Network

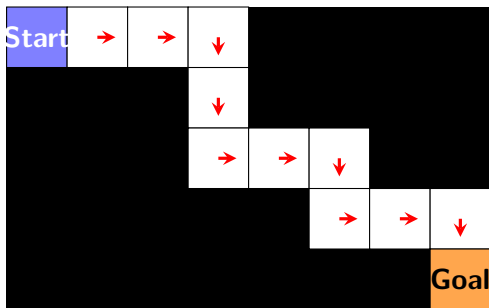
Agent–Environment Interaction



At each time t , the agent observes s_t , takes a_t , receives r_t , and the environment transitions to s_{t+1} .

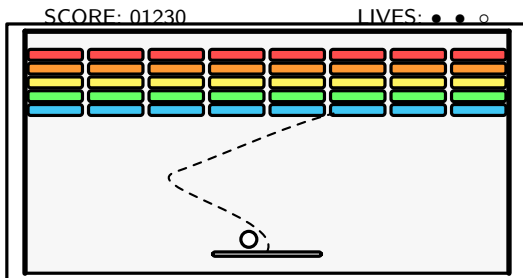
Examples: Maze-Navigating Agent

- ▶ **Environment:** Grid of free cells and walls; one start and one goal.
- ▶ **State:** Agent's coordinates in the grid.
- ▶ **Actions:** Move {up, down, left, right} if not blocked.
- ▶ **Reward:** +10 at goal; -1 per step; -5 for hitting a wall.



Examples: Atari Breakout

- ▶ **Environment:** The game screen with paddle, ball, bricks.
- ▶ **State:** Recent frames (image stack) capturing positions/velocities.
- ▶ **Actions:** {left, right, no-op} (sometimes fire).
- ▶ **Reward:** +1 per brick; +50 for clearing a level; -1 or termination on losing a life.



Supervised vs Unsupervised vs RL

Aspect	Supervised	Unsupervised	RL
Goal	Map inputs to known outputs	Discover structure	Maximize cumulative reward via interaction
Training Data	Labeled (x_i, y_i)	Unlabeled $\{x_i\}$	Interaction tuples (s_t, a_t, r_t, s_{t+1})
Feedback	Direct labels	None (implicit)	Delayed, evaluative (rewards)
Output	Label/value	Clusters / density	Policy $\pi^*(a \mid s)$ or $v^*(s)$

Mathematical Framework: Returns

Definition 1 (Discounted cumulative reward)

$$\begin{aligned} G_t &:= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\ &= r_t + \gamma G_{t+1}, \end{aligned}$$

with discount factor $0 \leq \gamma < 1$.

Remark 1 (Why discounting?)

Ensures finiteness: $\sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}$.

Goal: learn a policy that maximizes expected return.

Markov Decision Process (MDP)

Definition 2

An MDP is a 5-tuple (S, A, P, R, γ) :

- ▶ S : states, A : actions,
- ▶ $p(r, s' \mid s, a)$: transition probabilities,
- ▶ $r(s, a)$: immediate (expected) reward,
- ▶ $\gamma \in [0, 1)$: discount factor.

State-Value Function and Bellman Equation

Value (state-value) function:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \mid S_t = s],$$

where the expectation is taken over the joint distribution of all actions, states and rewards starting from t until the end of an episode.

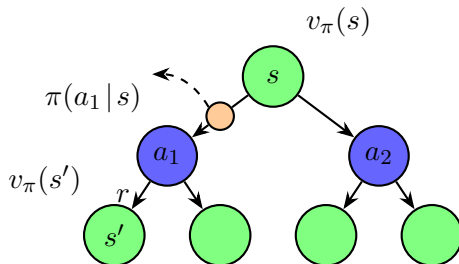
Bellman expectation equation:

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}_{\pi}[r_t + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_{t+1} = s']] \\ &= \sum_a \pi(a \mid s) \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')]. \end{aligned}$$

State-Value Function and Bellman Equation

Bellman expectation equation:

$$v_{\pi}(s_t) = \mathbb{E}_{\pi} [r_t + \gamma v_{\pi}(s_{t+1})]$$



Action-Value Function and Relations

Action-value function:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Bellman expectation (for q_{π}):

$$\begin{aligned} q_{\pi}(s, a) &= \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')] \\ &= \mathbb{E}[r_t + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t = a]. \end{aligned}$$

Relations:

$$q_{\pi}(s, a) = \sum_{r, s'} p(r, s' \mid s, a) [r + \gamma v_{\pi}(s')], \quad v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) q_{\pi}(s, a).$$

Learning Algorithms: Overview

Value Learning (Control):

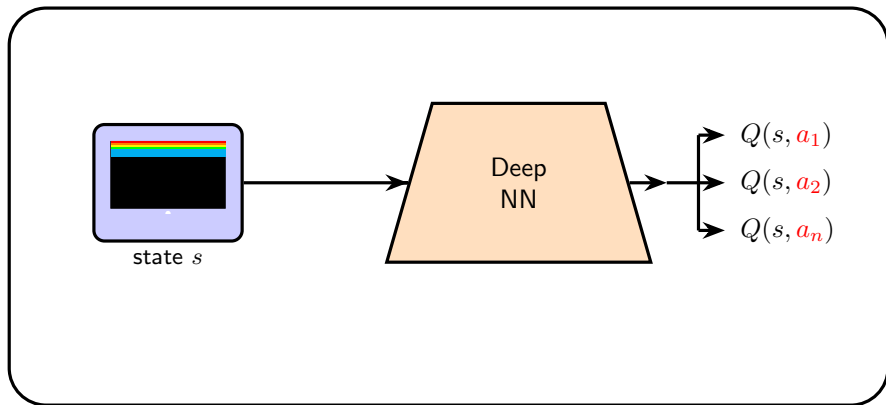
$$q_*(s, a) = \sum_{r, s'} p(r, s' \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$$

$$\pi_*(a \mid s) = \arg \max_a q_*(s, a).$$

Policy Learning (Policy Gradient):

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t], \quad \nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s) G_t].$$

Deep Q-Learning (DQN): Idea



Deep Q-Learning (DQN): Method

(1) Network architecture.

- ▶ Input: raw pixel observations
- ▶ Output: vector of Q-values, one per action.
- ▶ Typically a network with a few layers

(2) **Experience replay.** Transitions (s_t, a_t, r_t, s_{t+1}) are stored in a **replay memory**. During training, mini-batches of transitions are sampled uniformly at random from this memory to perform updates: $(s, a, r, s') \sim \mathcal{D}$.

(3) **Target network.** A separate **target network** $Q(s, a; \theta^-)$ is used to compute the target value:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-). \quad (1)$$

Deep Q-Learning (DQN): Method

The main network parameters θ are optimized to minimize the squared error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(y_t - Q(s, a; \theta))^2]. \quad (2)$$

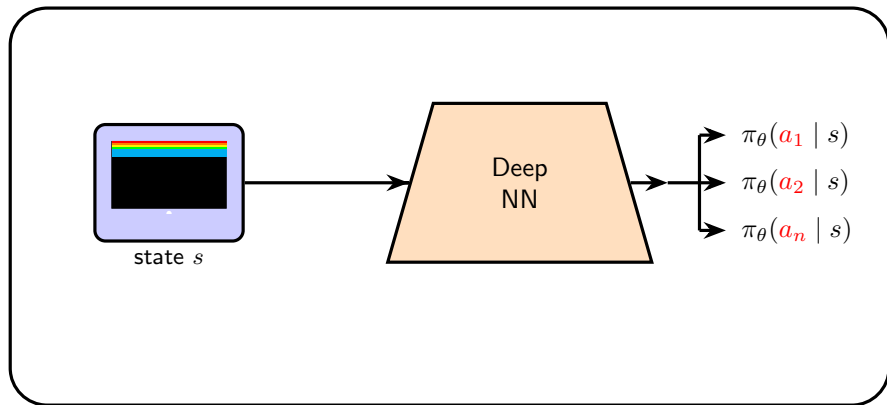
Every C steps, the target network is updated by copying the weights: $\theta^- \leftarrow \theta$.

(4) Optimization. SGD using the gradient:

$$\nabla_{\theta} L(\theta) = \mathbb{E}[(Q(s, a; \theta) - y_t) \nabla_{\theta} Q(s, a; \theta)].$$

(5) Exploration. DQN uses an ϵ -greedy exploration strategy: with probability ϵ , a random action is chosen, and with probability $1 - \epsilon$, the greedy action $\arg \max_a Q(s, a; \theta)$ is taken. The exploration rate ϵ decays over time.

Policy Learning



Policy Gradient Methods

Central idea Iteratively improve π_θ by:

1. collecting trajectories (state–action–reward sequences) generated by the current policy π_θ ,
2. estimating a gradient direction $\mathbf{g}$ that increases performance, and
3. updating the policy parameters according to $\theta \leftarrow \theta + \alpha \mathbf{g}$, where $\alpha > 0$ is the learning rate.

Policy Gradient Methods

Let $\tau = (s_0, a_0, s_1, \dots)$ be a random trajectory drawn from the policy with probability

$$p_{\theta}(\tau) := p(s_0) \prod_{t=0}^{\infty} \pi_{\theta}(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t).$$

Define the corresponding discounted reward by $R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)$. Formally, our objective is to find the parameters θ that maximize this expected return:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)] \\ &= \mathbb{E}_{s_t, a_t \sim \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]. \end{aligned} \tag{3}$$

Policy Gradient Methods

Consider

$$F(\theta) = \mathbb{E}_{\xi \sim p_{\theta}(\xi)}[f(\xi)].$$

The gradient of this parameterization is

$$\begin{aligned}\nabla_{\theta} F(\theta) &= \int f(\xi) \nabla_{\theta} p_{\theta}(\xi) d\xi \\ &= \int p_{\theta}(\xi) f(\xi) \nabla_{\theta} \log p_{\theta}(\xi) d\xi \\ &= \mathbb{E}_{\xi \sim p_{\theta}(\xi)}[f(\xi) \nabla_{\theta} \log p_{\theta}(\xi)].\end{aligned}$$

We conclude that

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a \mid s) R(\tau)]. \quad (4)$$

Policy Gradient Methods: REINFORCE algorithm [Wil92]

Require: initial policy parameter $\theta \in \mathbb{R}^d$, step size $\alpha > 0$, baseline $b(\cdot)$

- 1: **for** each episode **do**
- 2: Generate an episode $(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)$ following π_θ
- 3: **for** $t = 0$ to T **do**
- 4: $G_t \leftarrow \sum_{i=t}^T \gamma^{i-t} r_i$
- 5: $A_t \leftarrow G_t - b(s_t)$
- 6: $\theta \leftarrow \theta + \alpha A_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
- 7: **end for**
- 8: **end for**

REINFORCE: Two Examples

Bernoulli policy (binary action):

$$\pi_{\theta}(a \mid s) = p_{\theta}(s)^a [1 - p_{\theta}(s)]^{1-a}, \quad \nabla_{\theta} \log \pi_{\theta}(a \mid s) = (a - p_{\theta}(s)) \nabla_{\theta} z_{\theta}(s),$$

with $p_{\theta}(s) = \sigma(z_{\theta}(s))$. If $z_{\theta}(s) = \theta^{\top} \phi(s)$,

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [(a - p_{\theta}(s)) \phi(s) R(s, a)].$$

Two-layer policy network:

$$h_t = \sigma(W_1 x_t), \quad z_t = W_2 h_t, \quad p_t = \text{softmax}(z_t),$$

$$\nabla_{z_t} \log \pi_{\theta}(a_t \mid s_t) = y_t - p_t, \quad \nabla_{W_2} \log \pi_{\theta}(a_t \mid s_t) = (y_t - p_t) h_t^{\top}.$$

Actor-Critic: Combining Policy and Value Learning

Actor-Critic (AC) methods combine the strengths of policy gradients (actor) with value-function learning (critic).

- ▶ Actor is a differentiable policy $\pi_{\theta}(a \mid s)$,
- ▶ Critic is a value estimator $V_{\psi}(s) \approx v^{\pi_{\theta}}(s)$.

TD (Temporal-Difference) error and advantage. Given a transition (s_t, a_t, r_t, s_{t+1}) :

$$\delta_t = r_t + \gamma V_{\psi}(s_{t+1}) - V_{\psi}(s_t), \quad A_t := \delta_t.$$

$\leadsto A_t$ serves as a low-variance estimate of the advantage.

TD Error - Intuition

TD Error: it quantifies the difference between the estimated value of a state and a one-step updated estimate based on the next reward and next state.

- ▶ $V_\psi(s_t)$ is the current prediction of how good state s_t is.
- ▶ $r_t + \gamma V_\psi(s_{t+1})$ is a one-step bootstrap target.
- ▶ The TD error is their difference:

$$\text{TD error} = (\text{new target}) - (\text{old prediction}).$$

- ▶ If $\delta_t > 0$, the outcome was **better than expected** and $V_\psi(s_t)$ should increase.
- ▶ If $\delta_t < 0$, the outcome was **worse than expected** and $V_\psi(s_t)$ should decrease.

Actor-Critic Updates

At each step, the critic minimizes the squared TD error, while the actor performs a gradient ascent using the advantage estimate:

$$\begin{aligned}\psi &\leftarrow \psi - \beta \nabla_{\psi} \left(\frac{1}{2} \delta_t^2 \right) \\ \theta &\leftarrow \theta + \alpha A_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) + \lambda_H \nabla_{\theta} \mathcal{H}(\pi_{\theta}(\cdot \mid s_t)).\end{aligned}$$

α, β are learning rates, $\mathcal{H}$ is the entropy of the policy, and λ_H controls its weight (encouraging exploration).

Bias-variance tradeoff. Using $V_{\psi}(s_{t+1})$ to form the target (bootstrapping) reduces variance but introduces bias if V_{ψ} is inaccurate.

Multi-step Advantages and GAE

Variance can be reduced using multi-step or generalized advantage estimators.

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V_\psi(s_{t+n}),$$

$$A_t^{(n)} = G_t^{(n)} - V_\psi(s_t), \quad A_t^{\text{GAE}(\lambda)} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}, \quad \lambda \in [0, 1].$$

$\lambda \approx 1$: low bias, high variance (Monte Carlo). $\lambda = 0$: high bias, low variance (TD(0)). GAE interpolates between both extremes.

Relation to REINFORCE

REINFORCE uses the Monte Carlo return G_t as the learning signal. Actor-Critic replaces G_t by lower-variance (but possibly biased) advantage estimates.

This modification yields:

- ▶ Faster and more stable convergence
- ▶ Lower gradient variance
- ▶ Slight bias if V_ψ is imperfect

Implementation of the Critic

In practice, the critic $V_\psi(s)$ is implemented as a neural network mapping a state (or state-action pair) to a scalar prediction of expected return.

Typical structure:

$$V_\psi : \mathcal{S} \rightarrow \mathbb{R}, \quad V_\psi(s_t) \approx \mathbb{E}_{\pi_\theta}[G_t \mid s_t].$$

Actor and critic often share an encoder ϕ :

$$h_t = f_\phi(s_t), \quad \pi_\theta(a \mid s_t) = \text{softmax}(W_\pi h_t), \quad V_\psi(s_t) = W_V h_t.$$

This improves sample efficiency, while using separate output heads ensures stable optimization.



Simple statistical gradient-following algorithms for connectionist reinforcement learning.

Machine learning, 8(3):229–256, 1992.